Recommender Systems 1

Introduction to Data Mining

Seoul National University

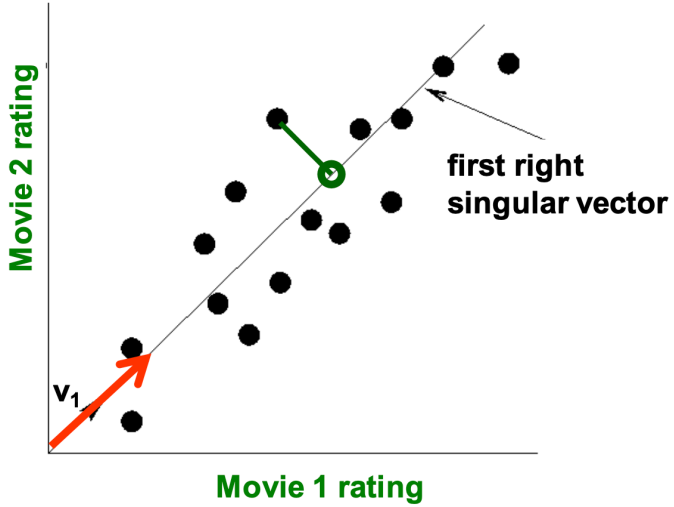
Spring 2026

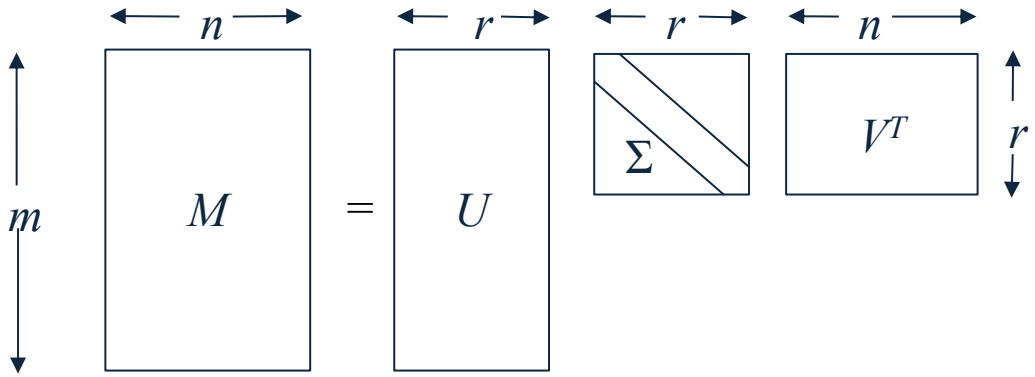
Announcements (Same as Before)

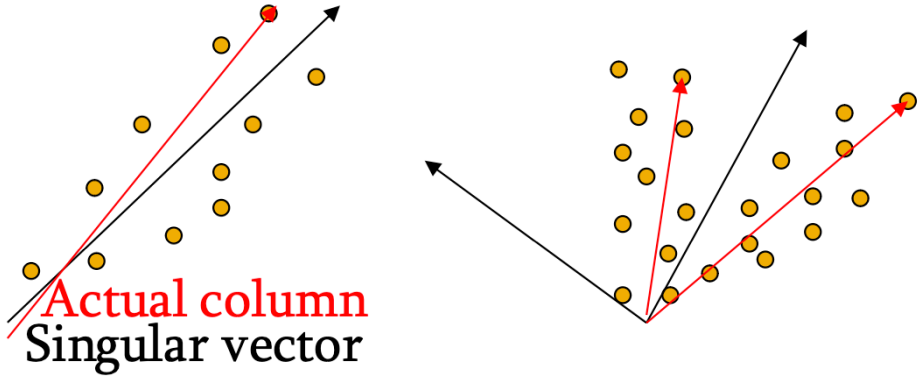
- **Assignments:**
  - HW1 (deadline: April 9)
  - HW2 (deadline: April 30)
- **Midterm:**
  - **When:** April 23 at 9:30 – 10:45
  - **Where:** 302-208 and 302-209 (students will be separated)
  - **How:** Closed-book; no calculators allowed

Recap

- Singular Value Decomposition
- Dimensionality Reduction with SVD
- CUR Decomposition







Outline

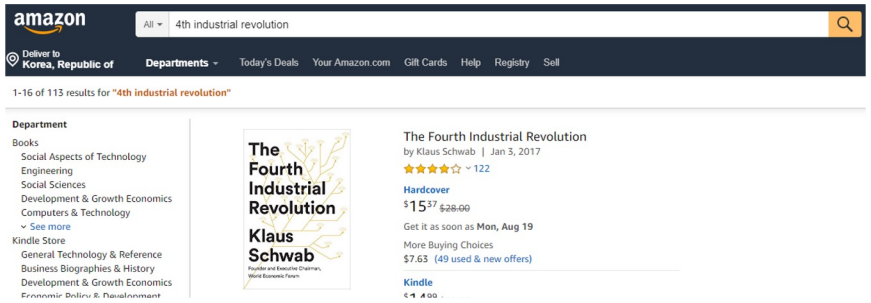
1. **<u>Recommender Systems</u>**
2. Content-based Recommendation
3. Collaborative Filtering
4. The Netflix Challenge

Recommender Systems

- Class of applications that predict user responses to options.
- **Non-personalized recommendations:**
  - *Editorial and hand-curated:* List of favorites, essential items, …
  - *Simple aggregates:* Top 10, most popular, recent uploads, …
- **Personalized recommendations:**
  - *Tailored to individual users:* Amazon, Netflix, YouTube, …

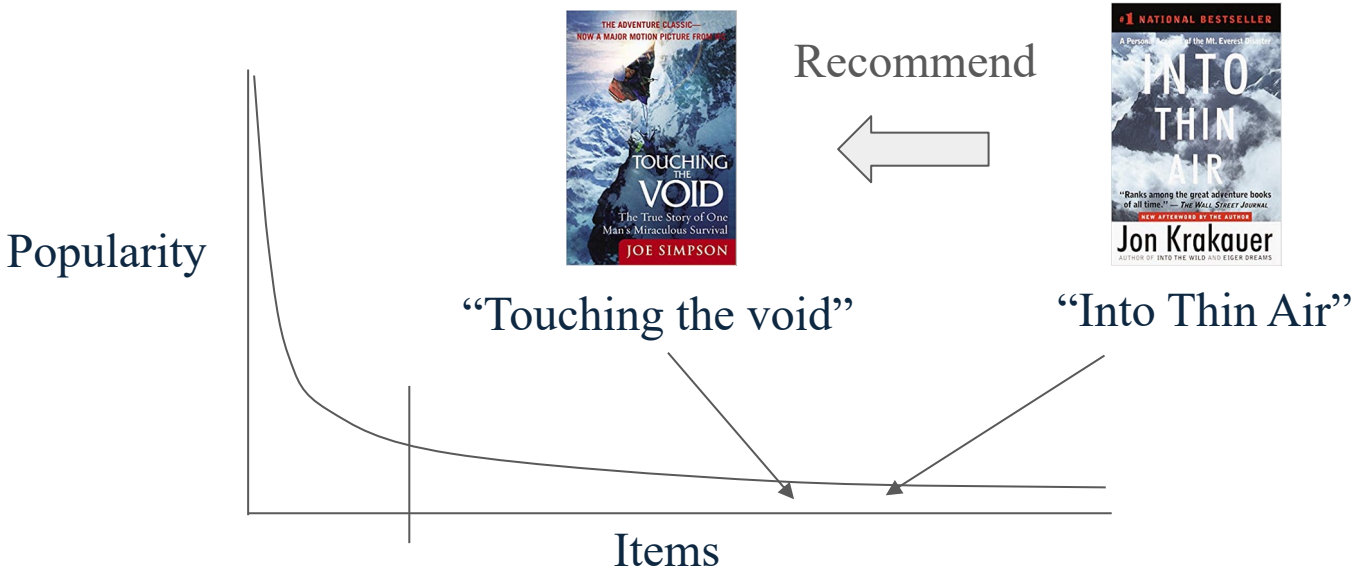
From Scarcity to Abundance

- Physical delivery systems have scarcity of resources.
  - Stores have limited shelf space to show to users.
- Online stores can make anything available to users.
  - Amazon offers millions of books.
  - Every user can have their own interface, i.e., a list of items.



The Long Tail

- Online institutions provide the entire range of items.
  - The **tail** as well as the **head** (popular) items
- Ideal recommendation is about finding proper tails.



Utility Matrix

- We consider two classes of entities: **users** and **items**.
- **Utility matrix** shows the preference of users for items.
  - The values come from an ordered set, e.g., 1-5 stars.
  - Assumed to be sparse, i.e., most entries are unknown.

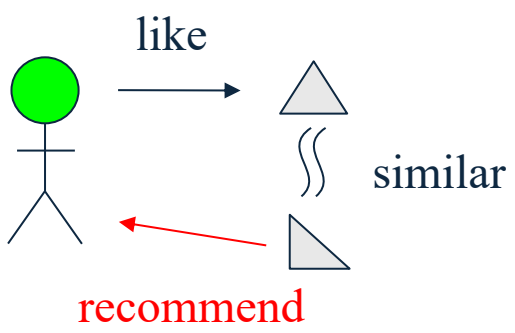
	HP1	HP2	HP3	TW	SW1	SW2	SW3
A	4			5	1		
B	5	5	4				
C				2	4	5	
D		3					3

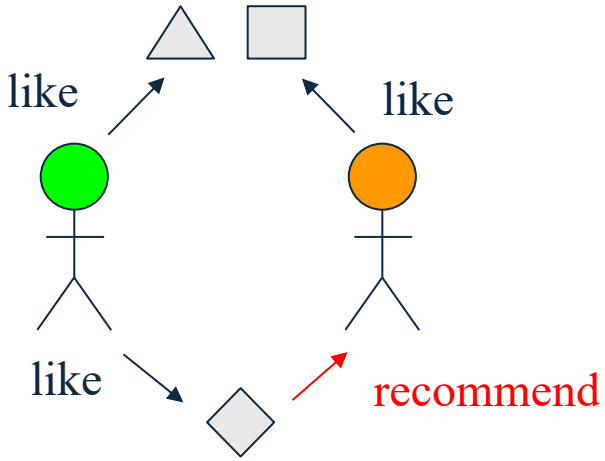
Gathering Ratings

- **Explicit feedback:** Ask users to rate items.
  - E.g., YouTube asks for likes/dislikes of watched videos.
  - Users are generally unwilling to provide responses.
  - *Biased* as it comes from people willing to provide ratings.
- **Implicit feedback:** Learn ratings from user actions.
  - If a user watches a movie, the user is said to “like” it.
  - Hard to model low ratings: 0 (no rating) or 1 (like).

Personalized Recommender Systems

- Two groups of recommender systems:
  - **Content-based:** Focus on the profiles (or features) of users/items.
  - **Collaborative filtering:** Focus on the interactions between users/items.





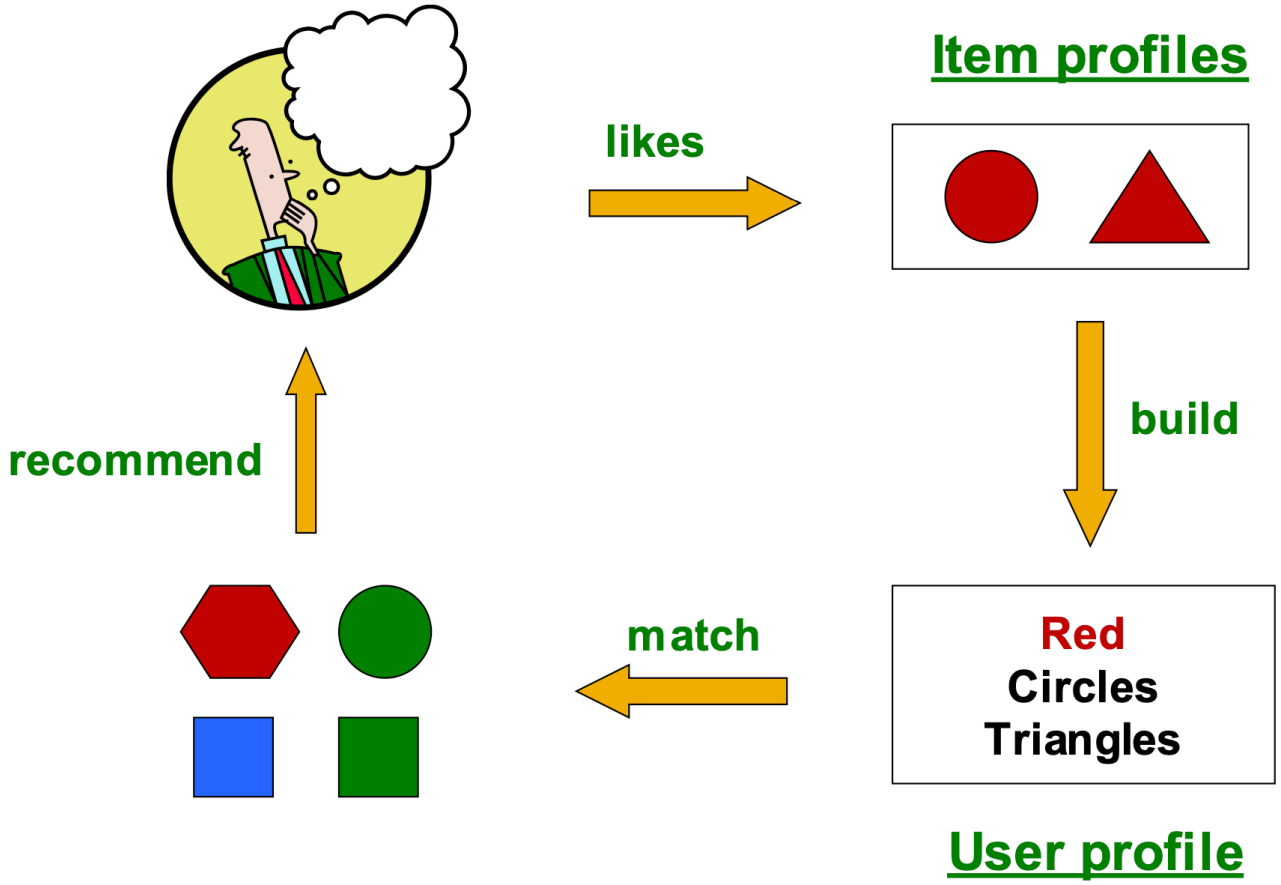
Outline

1. Recommender Systems
2. **<u>Content-based Recommendation</u>**
3. Collaborative Filtering
4. The Netflix Challenge

Content-based Recommendation

- **Main idea:** Use the profiles of items.
  - E.g., for a movie, { genre, director, actors, plot, release year }.
  - Recommend items similar to previous highly-rated items.
- **Examples:**
  - Recommend movies with same actor(s), director, genre, …
  - Recommend websites or blogs with “similar” content.

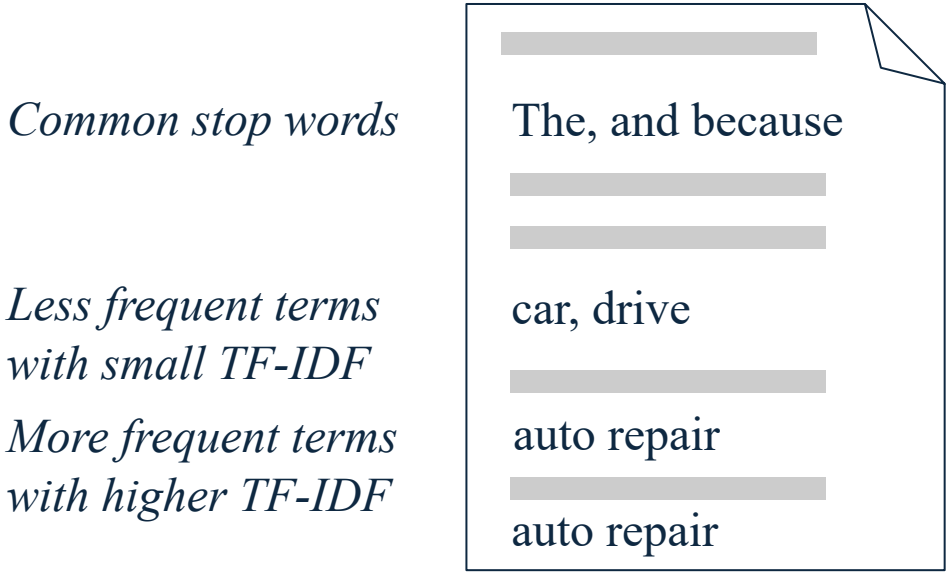
Plan of Action



Source: Stanford CS246 (2022)

Item Profiles

- For each item, create an **item profile** as a set of features.
- Finding features is not straightforward for some classes of items.
  - E.g., find a set of "important" words from a document.
  - Eliminate stop words.
  - Compute **TF-IDF scores** of words.
  - Use the top k words as its features.



Item Profiles as Vectors

- If a feature is a **discrete value**, use a **one-hot vector**.
  - Length L is the number of possible discrete values.
  - If it is a set of values, e.g., actors, it can be multi-hot.
- If a feature is **numerical**, use the (scaled) **exact value**.
  - Its scale might have to be adjusted considering other features.
  - E.g., average ratings for movies.

Example: Item Profiles

- We can also combine discrete and numerical features.
  - E.g., a set of actors, director, genre, and average rating.

Movie profiles:

	Actors								Director			Genre			Avg. rating
M_1:	0	1	1	0	1	1	0	1	0	1	0	0	1	0	2.5
M_2:	1	1	0	1	0	1	1	0	0	0	1	1	0	0	4.1

E.g., Julia Roberts

E.g., Science Fiction

User Profiles

- We need a **user profile** that summarizes the history of a user.
- Take some aggregation of item profiles using the utility matrix.
  - If the utility matrix is binary, the simple average is natural.
- Suppose that user U watched movies M_1 and M_2:

Movie M_1's profile:	0	1	1	0	1	1	0	0
Movie M_2's profile:	1	1	0	1	0	1	1	0
U's profile:	0.5	1	0.5	0.5	0.5	1	0.5	0

User Profiles

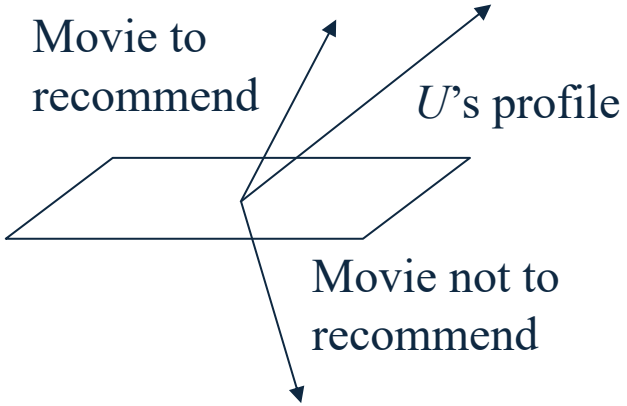
- If the matrix is not binary, weight item profiles by utility values.
  - E.g., say user U rates M_1 with rating 1 and M_2 with rating 4:

Movie M_1's profile:	0	1	1	0	1	1	0	0	Weight = 1
Movie M_2's profile:	1	1	0	1	0	1	1	0	Weight = 4
U's profile:	0.5	1	0.5	0.5	0.5	1	0.5	0	
U's weighted profile:	2	2.5	0.5	2	0.5	2.5	2	0	

Prediction Heuristic

- Compute the cosine distance between user and item profiles.
- For efficiency, one can apply LSH to quickly find candidate items.
  - Recall the random hyperplane approach.

U's profile:	0.5	1	-1	0.5	-1	1	0.5	-1.5
Movie to recommend:	0	1	0	0	0	1	0	0
Movie *not* to recommend:	0	0	1	0	1	0	0	0



Pros and Cons: Content-based Approach

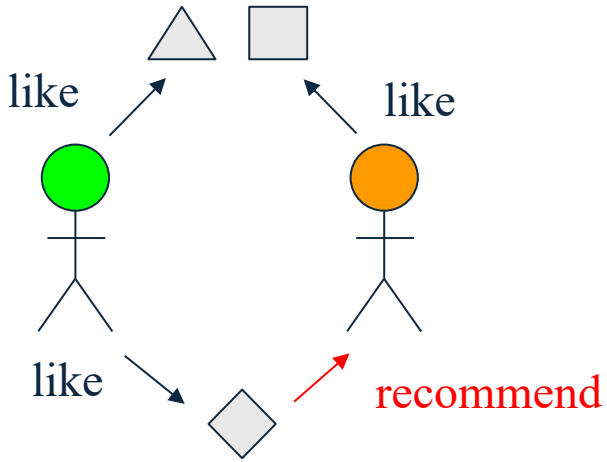
- **Pros:**
  - Able to recommend items to users with unique tastes.
  - Able to recommend new or unpopular items.
  - Able to provide explanations (by listing content features).
- **Cons:**
  - Finding the appropriate features may be difficult.
  - New users may not have a profile.
  - **Overspecialization:** Cannot recommend beyond user's profile.

Outline

1. Recommender Systems
2. Content-based Recommendation
3. **<u>Collaborative Filtering</u>**
4. The Netflix Challenge

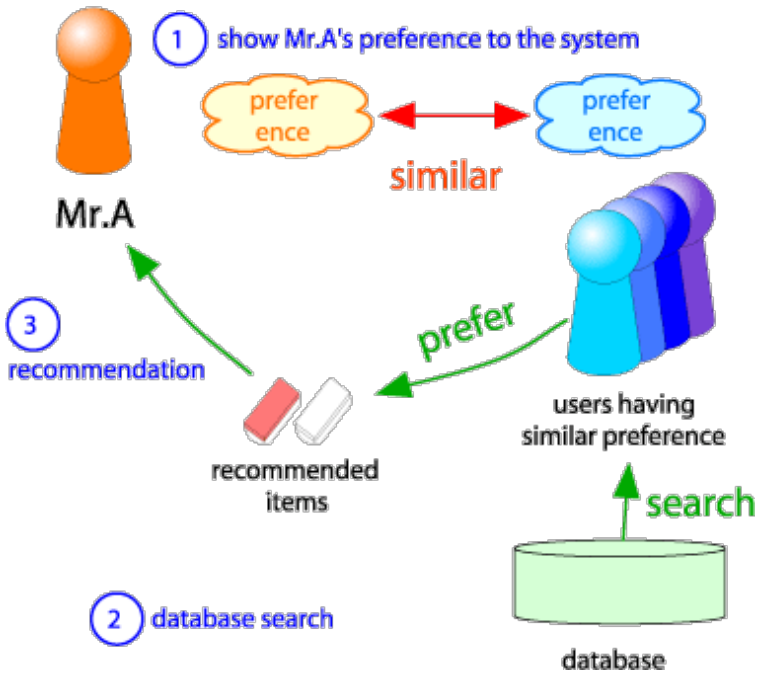
Collaborative Filtering

- Create user and item profiles solely from the utility matrix.
  - E.g., an item profile is the set of users who purchased it.
    $\Rightarrow$ the profile of item i is the i-th column of the matrix.
  - Don't care about the content of items.
- Comes in two flavors:
  - User-user collaborative filtering
  - Item-item collaborative filtering



User-User Collaborative Filtering

- Given user U, find users whose ratings are similar to U's ratings.
- Estimate U's ratings based on similar users.



Source: Stanford CS246 (2022)

Finding Similar Users

- There are various ways for defining which users are "similar."
- **Jaccard similarity:**
  - Treat ratings as sets, ignoring the values (i.e., likes vs. dislikes).
  - For this example, it seems intuitively wrong.
    - Since $\text{Jaccard}(A, B) = 1/5 < \text{Jaccard}(A, C) = 2/4$. Does it make sense?

	HP1	HP2	HP3	TW	SW1	SW2	SW3
A	4			5	1		
B	5	5	4				
C				2	4	5	
D		3					3

Finding Similar Users

- There are various ways for defining which users are "similar."
- **Cosine similarity:**
  - Treat ratings as points (or vectors), considering blanks as 0.
    - Questionable, since *no rating* doesn't mean *dislike*.
    - In this example, $v_A^\top v_B / \|v_A\| \|v_B\| = 0.380$ and $v_A^\top v_C / \|v_A\| \|v_C\| = 0.322$.

	HP1	HP2	HP3	TW	SW1	SW2	SW3
A	4			5	1		
B	5	5	4				
C				2	4	5	
D		3					3

Finding Similar Users

- There are various ways for defining which users are "similar."
- **Person's correlation coefficient:**
  - Consider only the ratings on items rated by both users.
  - $\mathrm{sim}(A, B) = \frac{\sum_{s \in S}(r_{A,s} - \bar{r}_A)(r_{B,s} - \bar{r}_B)}{\sqrt{\sum_{s \in S}(r_{A,s} - \bar{r}_A)^2}\sqrt{\sum_{s \in S}(r_{B,s} - \bar{r}_B)^2}}$
    - S: Items rated by both users A and B
    - $\bar{r}_A$ and $\bar{r}_B$: Average ratings of users A and B, resp.
  - **Limitation:** The size of S should be large enough.

Improving Similarity Metrics

- **How to improve Jaccard similarity:** Rounding the data.
  - Distinguish between high and low ratings.
  - Consider only the high ratings 3, 4, 5 as "1".
  - Then, $\text{Jaccard}(A, B) = 1/4 = \text{Jaccard}(A, C) = 0$.
    - Better than the previous result.

	HP1	HP2	HP3	TW	SW1	SW2	SW3
A	1			1			
B	1	1	1				
C					1	1	
D		1					1

Improving Similarity Metrics

- **How to improve cosine similarity:** Normalize ratings.
  - Subtract from each rating the average rating of that person.
  - Then, we have $\mathrm{sim}(A, B) = 0.092 > \mathrm{sim}(A, C) = -0.559$.
  - Notice that D's ratings are not worth taking seriously.
    - Does it make sense? What if the two ratings are both 5?

	HP1	HP2	HP3	TW	SW1	SW2	SW3
A	2/3			5/3	-7/3		
B	1/3	1/3	-2/3				
C				-5/3	1/3	4/3	
D		0					0

Rating Predictions

- **From similarity metrics to recommendations:**
  - Let r_x be the vector of user x's ratings.
  - Let N be the set of k users most similar to user x.
  - Let $N' \subseteq N$ be the subset of users who rated item i.
- Prediction for item i of user x:
  - **Simple version:** $r_{xi} = \frac{1}{k'} \sum_{y \in N'} r_{yi}$.
  - **Complicated version:** $r_{xi} = \frac{\sum_{y \in N'} \mathrm{sim}(x,y) \cdot r_{yi}}{\sum_{y \in N'} \mathrm{sim}(x,y)}$.

Item-Item Collaborative Filtering

- **Another view: Item-item collaborative filtering.**
  - For item i, find other similar items.
  - Estimate rating for item i based on ratings for similar items.
  - Can use the same similarity metrics as in the user-user model.

$$r_{xi} = \frac{\sum_{j \in N(i;x)} \mathrm{sim}(i,j) \cdot r_{xj}}{\sum_{j \in N(i;x)} \mathrm{sim}(i,j)}$$

  - $N(i;x)$ is the set of items similar to item i and rated by user x.

Item-Item vs. User-User

- **Item-item similarity is often more reliable.**
  - Intuitively, items are classifiable in simple terms, e.g., one genre.
  - Users may like multiple genres, so harder to compute similarity.
- However, there is no clear advantage of one from another.
  - E.g., user-user is better for relatively new items.

	HP1	HP2	HP3	TW	SW1	SW2	SW3
A	4			5	1		
B	5	5	4				
C				2	4	5	
D		3					3

Pros and Cons: Collaborative Filtering

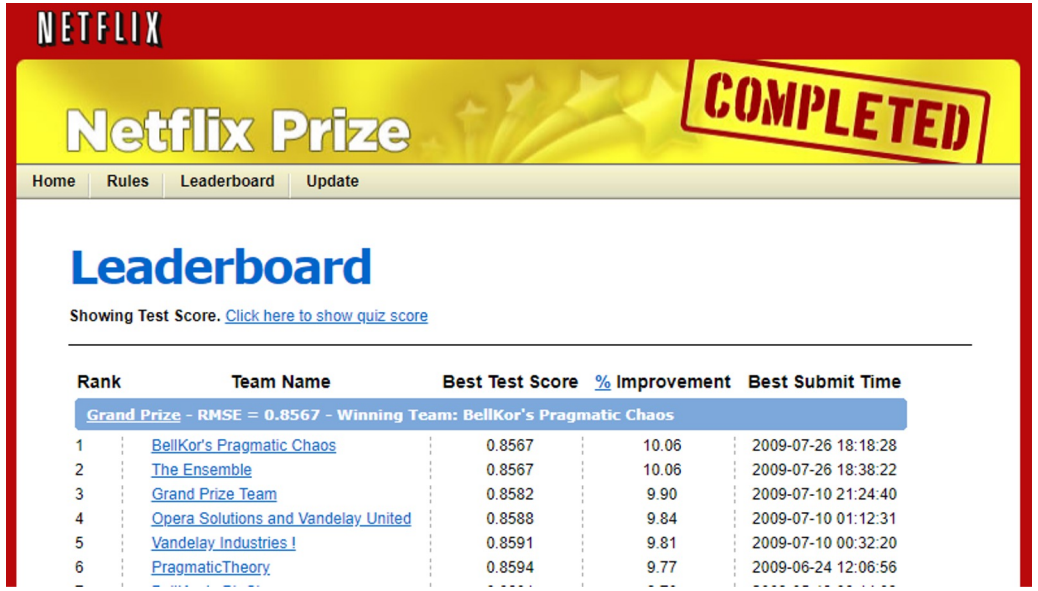
- **Pros:**
  - Do not have to come up with features (or profiles).
- **Cons:**
  - Need enough users in the system to find a match.
  - Cannot recommend new or unpopular items that have not been rated.
  - Cannot recommend items to someone with unique taste.
    - I.e., tends to recommend popular items.

Outline

1. Recommender Systems
2. Content-based Recommendation
3. Collaborative Filtering
4. **<u>The Netflix Challenge</u>**

The Netflix Challenge (2006)

- Netflix offered $1M to beat its movie recommendation algorithm.
- The prize was awarded after three years of work.

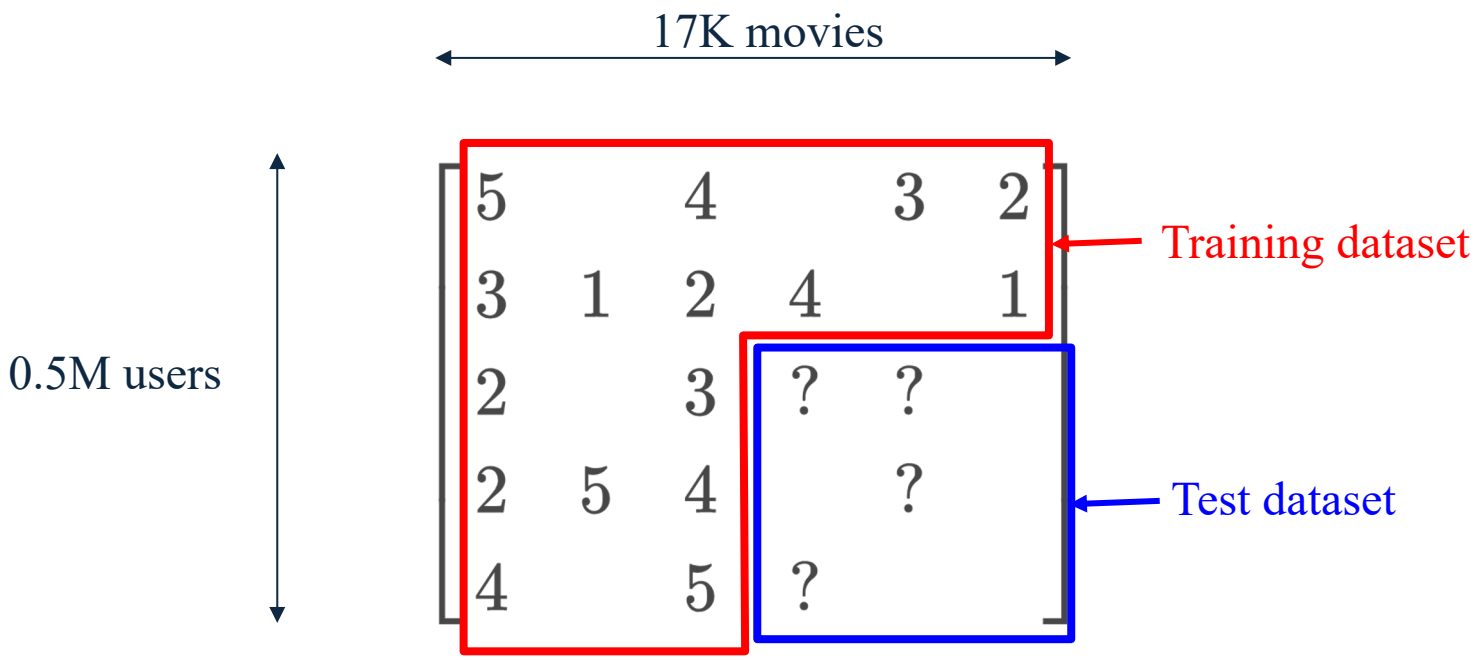


Description of the Challenge

- **Training data:**
  - 100M 1-5 ratings from 480K users and on 18K movies.
  - Collected for 6 years (2000 to 2005).
- **Test data:**
  - Set of the last few ratings of each user (2.8M in total).
  - Evaluation criterion: Root mean square error (RMSE).
  - Netflix's system RMSE: 0.9514.
- **Competition:** 2,700+ teams.

Utility Matrix from Netflix

- **Goal:** Minimize RMSE on the test data.
  - $\text{RMSE} = \text{sqrt}\left(\frac{1}{|T|}\sum_{(x,i)\in T}(r_{xi} - \hat{r}_{xi})^2\right)$ where T is the test set.



Training and Test Data

- **Test data:** Notable difference from the previous topics.
- Goal of data mining is often to find **patterns in given data**.
  - Clustering, frequent itemsets, dimensionality reduction.
- The Netflix challenge is about **generalizing to unseen data**.
  - Most machine learning problems are this case.
  - Focus on performance, rather than scalability or running time.

Key Takeaways

- Not easy to outperform very simple approaches.
- **Naïve approach:** Predict the rating r_{ui} as $(\mu_u + \mu_i)/2$.
  - μ_u = The average rating of user u.
  - μ_i = The average rating for item i by all users.
- CineMatch was **only 3% better** than the naïve approach.
  - CineMatch: Netflix's previous solution.

Key Takeaways

- Additional data, other than the utility matrix, can help or not.
- **Movie information:**
  - IMDB to get information of actors, directors, and genres.
  - Not effective, maybe because they didn’t give useful information.
- **Time information:**
  - Decreasing vs. increasing reputation over time.
  - Examined the ratings of a movie to see its rating trend in time.

Key Takeaways

- The winning solution was a combination of several algorithms.
  - These algorithms were developed independently.
  - The second team also used a blend of independent algorithms.
- Combining different algorithms is effective in general.
  - Called an **ensemble** approach.
  - **Cons:** Difficult interpretation, extensive computation, etc.

Summary

1. Recommender Systems
   - The long tail phenomenon
   - Utility matrix
2. Content-based Recommendation
   - User / item profiles
3. Collaborative Filtering
   - Defining “similarity”
4. The Netflix Challenge
   - Key takeaways